

Kit de recursos educativos del módulo 2

PF1822 · ESPECIALIZACIÓN EN DESARROLLO CON IA
Módulo 2: MA04576 INTRODUCCIÓN A MODELOS GENERATIVOS Y CONSUMO POR API · 21 h

1 Índice del kit	6 Indicadores de logro
2 Bienvenida e infografía	7 Instrumentos de evaluación
3 Cápsulas y cuadro comparativo	8 Portafolio
4 Herramientas didácticas	9 Retroalimentación
5 Actividades prácticas	10 Metodología, motivación y siglo XXI

Para: Natalia — hackea.pro

Fecha: 24 de septiembre de 2026

Estado: borrador para revisión

PF1822 · Módulo 2: kit de recursos educativos

Módulo: MA04576 INTRODUCCIÓN A MODELOS GENERATIVOS Y CONSUMO POR API · 21 h · e-learning

Estado: borrador · **Producido:** 2026-09-24 (sesión 2026-09-24-claude-code-05)

Qué recursos lleva el módulo 2 montado en el LMS, a qué exigencia responde cada uno y dónde está su contenido desarrollado. Las exigencias salen del Anexo N°7, numeral 7 (págs. 109–111): dos actividades prácticas con respuesta modelada; dos herramientas didácticas efectivas (tutoriales, videos interactivos, audios, multimedia); medios como presentaciones, instructivos y cuadros comparativos; un ambiente con íconos, multimedia e imágenes que motive; y tres estrategias de habilidades del siglo XXI de la lista oficial.

Hilo conductor del módulo

Todo el módulo trabaja sobre **un solo caso**: *Nube Sur*, una empresa de software **ficticia** cuya mesa de ayuda recibe cientos de tickets al día, largos, desordenados, con HTML de correo y firmas. El jefe de soporte necesita que cada ticket llegue con un resumen de una línea para priorizarlo. En el módulo el participante construye ese **resumidor de tickets**: diseña su arquitectura (AE1), programa el cliente que consume el modelo por API (AE2), escribe y afina los prompts (AE3) y arma el pipeline que limpia los tickets y mide la calidad de los resúmenes (AE4).

Nube Sur es inventada. No corresponde a ninguna institución ni empresa real, y este contenido es común a todas las instituciones que presenten el curso.

Ruta del módulo (propuesta de distribución de las 21 h)

Tramo	Aprendizaje esperado	Horas	Sincrónico	Producto del tramo
1	AE1 · modelos generativos y arquitectura	4 h	1,5 h	Diagrama de arquitectura del resumidor + cuadro de tipos de modelos
2	AE2 · consumo de API REST	6 h	1,5 h	Clase <code>ClienteIA</code> con docstrings y pruebas
3	AE3 · diseño de prompts	5 h	1,5 h	Prompts zero-shot y few-shot con registro de iteraciones
4	AE4 · preprocesamiento y evaluación	6 h	1,5 h	Pipeline de limpieza + tabla de métricas ROUGE y BLEU
Total		21 h	6 h	

PENDIENTE: la distribución de horas por aprendizaje la propone este kit; SIPFOR solo fija las 21 h del módulo. La valida Natalia.

Los recursos

Id	Recurso	Formato en el LMS	Exigencia	Archivo
R01	Video de bienvenida	Video 1:40 con avatar o docente	C3 motivación	R-bienvenida-e-infografia.md
R02	Infografía "Ruta del módulo"	Imagen descargable + versión accesible en texto	C3 · medio	R-bienvenida-e-infografia.md
R03	Cuatro cápsulas de contenido, una por AE	Presentaciones narradas (8–10 láminas)	C4 d) presentaciones	R-capsulas.md
R04	Cuadro comparativo de modelos generativos	Tabla interactiva o PDF	C4 d) cuadro comparativo · AE1	R-capsulas.md

Id	Recurso	Formato en el LMS	Exigencia	Archivo
R05	Herramienta didáctica 1: notebook guiado "Tu primera llamada a un modelo"	Notebook (Colab o JupyterLab) con celdas de autocomprobación	C4	C4-herramientas-didacticas.md
R06	Herramienta didáctica 2: video interactivo "Del prompt a la respuesta"	Video con preguntas incrustadas (H5P)	C4	C4-herramientas-didacticas.md
R07	Actividad práctica 1: "Un cliente de API para el resumidor"	Entrega de código con pruebas	C2 · resolución de problemas	C2-actividades.md
R08	Actividad práctica 2: "Laboratorio de prompts y métricas"	Simulación con tablero de puntajes (gamificación)	C2 · análisis de caso	C2-actividades.md
R09	Indicadores de logro	Tabla en la guía del participante	B1	B1-indicadores.md
R10	Tres instrumentos de evaluación	Rúbrica, proyecto de desempeño, prueba objetiva	B2	B2-instrumentos.md
R11	Portafolio con sus 6 elementos + rúbrica	Sitio publicado + repositorio	B3	B3-portafolio.md
R12	Feedback, autoevaluación, coevaluación y bitácora	Formularios y plantilla en el LMS	B4	B4-retroalimentacion.md
R13	Estrategia metodológica, motivación y siglo XXI	Texto del Anexo 2, sección VI	C1 · C3 · C5	C-metodologia.md

Qué recurso cubre qué aprendizaje

Recurso	AE1	AE2	AE3	AE4
R01 Bienvenida	•			
R02 Infografía	•	•	•	•
R03 Cápsulas	•	•	•	•
R04 Cuadro comparativo	•			
R05 Notebook guiado		•	•	•
R06 Video interactivo		•	•	
R07 Actividad 1	•	•	•	
R08 Actividad 2			•	•
R10 Instrumentos (los tres)	•	•	•	•
R11 Portafolio	•	•	•	•

Lo que necesita cada participante para usar el kit

Según los recursos del módulo en SIPFOR: equipo con al menos 8 GB de RAM y 500 GB de disco, conexión a internet, **Python 3.10 o superior**, Visual Studio Code o JupyterLab, entornos virtuales, Git y GitHub, un cliente HTTP (Postman o equivalente) y, como alternativa, Google Colab, una máquina virtual o un contenedor.

Bibliotecas que usa el kit: `httpx` (o `requests`), `pytest`, `spacy` con el modelo `es_core_news_sm`, `nltk`, `rouge-score` y `scikit-learn`.

PENDIENTE: **acceso a las API de OpenAI y Hugging Face** para cada participante (el AE2 las nombra): quién provee las claves de práctica y con qué presupuesto. Lo decide cada institución; va en la sección VIII e) del Anexo 2. Las actividades se pueden ensayar sin clave con la API simulada de las pruebas, pero la práctica real necesita acceso.

PF1822 · Módulo 2 · R01 y R02 — Video de bienvenida e infografía

Estado: borrador · **Exigencia:** C3 aspectos motivacionales (Anexo N°7, num. 7 c): un ambiente intuitivo, con íconos, multimedia e imágenes, que invite a avanzar.

R01 · Video de bienvenida (1 minuto 40 segundos)

Formato: presentador (docente o avatar) y pantalla con código. Primera pieza del módulo en el LMS. Subtítulos obligatorios. **Tono:** de colega desarrollador; muestra antes de explicar.

Tiempo	Imagen	Locución
0:00–0:10	Presentador a cámara	"Hola. Bienvenida, bienvenido al módulo 2: Introducción a modelos generativos y consumo por API."
0:10–0:30	Una bandeja con cientos de tickets de soporte, largos y con firmas	"Imagina la mesa de ayuda de una empresa de software que recibe cientos de tickets al día. Nadie alcanza a leerlos todos antes de priorizar. La empresa se llama Nube Sur, es inventada, y va a ser tu cliente durante el módulo."
0:30–0:55	Terminal: se ejecuta un script, entra un ticket y sale un resumen de una línea	"En estas 21 horas vas a construir su resumidor de tickets: una aplicación que limpia cada ticket, le pide a un modelo de lenguaje un resumen por API y mide si el resumen es bueno."
0:55–1:15	Infografía de la ruta: cuatro estaciones	"Cuatro estaciones: entender los modelos y diseñar la arquitectura, programar el cliente de la API, escribir buenos prompts, y limpiar y medir texto. Cada estación deja código en tu portafolio."
1:15–1:30	Tablero del laboratorio con puntajes ROUGE e insignias	"Al final, el laboratorio: pruebas estrategias de prompt, las mides y compites en el tablero por el mejor resumen. Pero ojo: gana quien decide con evidencia."
1:30–1:40	Presentador; botón "Empezar estación 1"	"Primer paso: mira la ruta, completa el autodiagnóstico y abre el notebook guiado. Nos vemos en la primera sesión en vivo."

Notas de producción: si se usa avatar, una pieza de 60 a 120 segundos por módulo; no reemplaza al tutor. Placa final con nombre del tutor y horarios, que completa cada institución. **PENDIENTE:** nombre del tutor y horarios, por institución.

R02 · Infografía "Ruta del módulo 2"

Formato: imagen vertical (1080 × 1920 px), descargable en PDF, con esta versión en texto para lectores de pantalla. Un ícono por estación, repetido en los títulos del LMS.

Encabezado

MÓDULO 2 · INTRODUCCIÓN A MODELOS GENERATIVOS Y CONSUMO POR API · 21 h *Tu misión: construir el resumidor de tickets de Nube Sur.*

Bloque "Al terminar serás capaz de" (competencia del módulo, textual):

IMPLEMENTAR LA ARQUITECTURA BÁSICA DE UNA APLICACIÓN QUE INTEGRE MODELOS DE IA MEDIANTE APIS Y PREPROCESAMIENTO DE TEXTO PARA CONSUMO POR MODELOS DE LENGUAJE, GARANTIZANDO EL CONSUMO EFICIENTE Y LA EVALUACIÓN DE RESULTADOS EN UN ENTORNO DE DESARROLLO DE SOFTWARE

Bloque "Tu ruta"

Estación	Ícono	Título	Qué haces	Qué te llevas	Horas
1	Plano con cajas conectadas	Entender y diseñar	Conoces los tipos de modelos y dibujas la arquitectura del resumidor	Cuadro de modelos + diagrama	4 h
2	Llave y flecha hacia una nube	Conectar	Programas ClienteIA: GET, POST, embeddings, pruebas	Módulo Python con pruebas en verde	6 h
3	Globo de diálogo con ejemplos	Pedir bien	Prompts zero-shot y few-shot, y el efecto de temperature	Registro de iteraciones	5 h
4	Embudo y gráfico de barras	Limpiar y medir	Pipeline de limpieza y métricas ROUGE y BLEU	Tabla de resultados + recomendación	6 h

Bloque "Cómo te acompañamos"

- 4 sesiones en vivo de 1,5 h, una por estación. **PENDIENTE:** días y horarios por institución.
- Revisión de tu código en máximo 2 días hábiles.
- Foro de dudas por estación, respondido por el tutor.

Bloque "Cómo te evaluamos"

- Rúbrica de tu solución en las actividades 1 y 2 (35 %).
- Proyecto "Asistente de preguntas frecuentes" al cierre (45 %).
- Prueba de conceptos y lectura de código (20 %).
- Tu portafolio y tu repositorio quedan publicados para mostrar a un empleador.

Pie

Herramientas del módulo: Python · httpx · pytest · spaCy · NLTK · rouge-score · Git y GitHub · API de OpenAI y Hugging Face. Regla de oro: tu clave nunca va en el código. Empieza aquí → autodiagnóstico + notebook guiado.

PF1822 · Módulo 2 · R03 y R04 — Cápsulas de contenido y cuadro comparativo

Estado: borrador · **Exigencia:** C4 d) uso de los medios (Anexo N°7, num. 7): presentaciones y cuadro comparativo como medios para transferir el aprendizaje.

Formato de cada cápsula: presentación narrada de 8 a 12 minutos (8–10 láminas), con transcripción. Una por aprendizaje esperado. Contenidos tomados de la ficha SIPFOR del módulo (00-ficha-sipfor.md).

Cápsula 1 · Modelos generativos y su ecosistema (AE1 · 1.1, 1.2, 1.3)

#	Lámina	Contenido
1	Portada	"Qué hace un modelo generativo y dónde encaja en una aplicación". El caso: el resumidor de tickets de Nube Sur.
2	IA generativa frente a IA tradicional	La IA tradicional basada en reglas decide con condiciones escritas por personas; la generativa produce contenido nuevo (texto, código, imágenes) aprendido de datos.
3	Tipos por lo que generan	Texto, imagen, audio y video.
4	Familias técnicas	Autorregresivos: generan token a token (texto y código). De difusión: parten de ruido y lo refinan (imágenes, audio). Transformers: la arquitectura detrás de la mayoría de los modelos de lenguaje actuales. Ver el cuadro R04.
5	Casos de uso	Generación de texto y de código, síntesis de imágenes, preguntas y respuestas, resúmenes, generación de pruebas.
6	Arquitecturas	Solicitud-respuesta (una llamada, una respuesta) y pipelines (varias etapas encadenadas: limpiar → generar → evaluar → guardar).
7	Componentes	Cliente · servidor API · módulo de preprocesamiento · modelo de IA · base de datos o <i>vector store</i> . Flujo de datos de extremo a extremo con el ticket de ejemplo.
8	Tecnologías y bibliotecas	API de OpenAI · Hugging Face Transformers · TensorFlow · PyTorch · LangChain · Diffusers: qué resuelve cada una y en qué módulo del plan se vuelve a ver.
9	Elegir una arquitectura	Criterios: escalabilidad (¿aguanta 10 veces más tickets?), seguridad (¿dónde están la clave y los datos personales?), costo (tokens por ticket) y mantenimiento (¿se puede cambiar de modelo sin reescribir todo?).
10	Tu tarea	Diagrama del resumidor y su justificación: actividad 1, parte A.

Cápsula 2 · Consumir modelos por API (AE2 · 2.1, 2.2, 2.3)

#	Lámina	Contenido
1	Portada	"Hablar con un modelo es hacer una solicitud HTTP bien hecha".
2	REST en dos métodos	GET para leer (listar modelos); POST para enviar datos y pedir un resultado (generar, obtener embeddings).
3	Leer la documentación oficial	Cómo están organizados los manuales de OpenAI y de Hugging Face: referencia de la API, guías, ejemplos. En cada endpoint buscar URL, autenticación, cuerpo y respuesta.
4	Endpoints de generación y de embeddings	Generación: <code>messages</code> con roles → texto en <code>choices</code> y consumo en <code>usage</code> . Embeddings: <code>input</code> → un vector por texto en <code>data</code> .
5	Autenticación y variables de entorno	Encabezado <code>Authorization: Bearer <clave></code> ; la clave vive en una variable de entorno o en los <i>Secrets</i> del entorno, nunca en el código ni en el repositorio.

#	Lámina	Contenido
6	requests y httpx	Las dos bibliotecas hacen lo mismo en lo básico; <code>httpx</code> suma tiempos de espera claros y una forma sencilla de simular la API en pruebas.
7	Cuerpos JSON y lectura de la respuesta	Armar el diccionario, enviarlo con <code>json=</code> , leer <code>respuesta.json()</code> . Siempre mirar primero <code>status_code</code> .
8	Errores del servicio	401 clave inválida · 429 demasiadas solicitudes (esperar y reintentar) · 5xx falla del proveedor. Un mensaje claro vale más que un programa que se cae.
9	Modularizar, documentar y probar	Una clase que encapsula las llamadas, con docstrings, y pruebas unitarias que simulan la API para correr sin conexión ni costo.
10	Tu tarea	Notebook guiado R05 y actividad 1, parte B: <code>ClienteIA</code> .

OpenAI y Hugging Face en una tabla (para la lámina 3):

	API de OpenAI	Hugging Face
Qué ofrece	Modelos propios de la empresa por API	Un repositorio de miles de modelos abiertos y servicios para ejecutarlos (por API o en la propia máquina)
Autenticación	Clave de API en el encabezado	Token de acceso en el encabezado
Cuándo conviene	Calidad alta con poca configuración	Elegir o cambiar de modelo, usar modelos abiertos, ejecutar localmente

PENDIENTE: el tutor revisa, antes de publicar, las URL y los nombres de modelos vigentes en ambas documentaciones; cambian con frecuencia.

Cápsula 3 · Diseño de prompts (AE3 · 3.1, 3.2, 3.3)

#	Lámina	Contenido
1	Portada	"El prompt es la especificación".
2	Qué es un prompt	La entrada que le dice al modelo qué hacer. Estructura: mensaje de sistema (rol y reglas) y mensaje de usuario (la tarea y los datos).
3	Tipos de prompting	Zero-shot : solo instrucciones. Few-shot : instrucciones más ejemplos de entrada y salida. Instruccional : pasos explícitos a seguir.
4	Buenas prácticas	Claridad, rol definido, contexto suficiente, restricciones explícitas, evitar ambigüedad.
5	Formato de salida	Pedir listas, JSON o tablas cuando otro programa va a leer la respuesta.
6	Ejemplos prácticos	Un prompt para generar código, uno para resumir, uno para validar y uno para reformular, con su salida.
7	Parámetros	temperature (variedad), max_tokens (largo máximo), top_p (acota las opciones). Para resúmenes, temperature baja.
8	Limitaciones	Alucinaciones, falta de control, sensibilidad a la redacción: un cambio pequeño de palabras cambia la respuesta.
9	Iterar	Cambiar una cosa a la vez, anotar qué cambió y qué efecto tuvo, y decidir con los resultados.
10	Tu tarea	Video interactivo R06 y actividad 2, pasos 2 y 3.

Cápsula 4 · Preparar y medir texto (AE4 · 4.1, 4.2, 4.3)

#	Lámina	Contenido
1	Portada	"Basura entra, basura sale: limpiar antes, medir después".

#	Lámina	Contenido
2	Conceptos básicos de NLP	Token (unidad que procesa el modelo), palabra, lema (forma de diccionario), stopwords (palabras frecuentes con poco contenido), n-gramas (secuencias de n tokens).
3	Limpieza	Quitar HTML, caracteres especiales, firmas y ruido; reemplazar correos y enlaces por marcas. Cuidado con no borrar el problema que reporta el ticket.
4	Normalización	Mayúsculas y minúsculas, espacios, estandarización de texto; tildes solo para comparar al medir.
5	Tokenizar, lematizar, stemming	spaCy (<code>es_core_news_sm</code>) y NLTK. Lematizar: "corriendo" → "correr"; stemming: recorta la raíz.
6	Vectorización básica	CountVectorizer y TF-IDF de scikit-learn: representar textos como números para compararlos.
7	Un pipeline modular	Funciones reutilizables por etapa (<code>limpiar</code> , <code>tokenizar</code> , <code>ngramas</code> , <code>medir</code>) que se encadenan y se prueban por separado.
8	Métricas de calidad	ROUGE (cuánto de la referencia aparece en el resumen), BLEU (cuánto del resumen aparece en la referencia, con penalización por largo), coherencia y relevancia con pauta humana. En español, usar un tokenizador que respete las tildes.
9	Registrar y ajustar	Una fila por prueba en un CSV; identificar ajustes a partir de los números.
10	Tu tarea	Actividad 2, pasos 1, 4 y 5.

R04 · Cuadro comparativo: familias de modelos generativos

AE1 · indicador 1.1. En la actividad 1 el participante agrega una columna con un caso de uso para Nube Sur en cada fila.

Familia	Cómo genera	Salida típica	Casos de uso en una organización	Ejemplos de tecnologías	A considerar
Autorregresivos	Predicen el siguiente token a partir de los anteriores, uno a la vez	Texto, código	Resúmenes, respuestas a clientes, generación de código y de pruebas	Modelos de lenguaje tipo GPT, Llama o Mistral	Pueden inventar datos; el costo depende de los tokens
De difusión	Parten de ruido y lo refinan en muchos pasos hasta llegar a la salida	Imágenes, audio	Imágenes para marketing, prototipos visuales	Stable Diffusion, biblioteca Diffusers	Más cómputo por resultado; derechos de uso de las imágenes
Transformers codificadores	Convierten un texto en una representación numérica; no generan texto libre	Embeddings, clasificación	Buscar tickets parecidos, clasificar por tema	Familia BERT, modelos de embeddings	Base de la búsqueda semántica del módulo 3 del plan
Transformers codificador-decodificador	Leen una entrada completa y generan una salida transformada	Texto a texto, audio a texto	Traducción, resumen, transcripción	Familias T5 y BART, Whisper	Buenos para tareas acotadas de transformación

Nota para el tutor: "autorregresivo" y "de difusión" describen **cómo se genera**; "transformer" describe **la arquitectura**. La mayoría de los modelos de lenguaje actuales son transformers autorregresivos. El plan los nombra como tres tipos: conviene explicar esta superposición en la lámina 4 para que la clasificación no confunda.

PF1822 · Módulo 2 · C4 — Herramientas didácticas

Estado: borrador · **Va en:** Anexo N°2, sección VI c), **con el enlace al LMS**

Guía: Anexo N°7, num. 7 · **Para el 7,0:** 2 herramientas distintas y **ambas** efectivas para adquirir la habilidad. Si solo una lo es, la nota baja a 5,0.

1. **Notebook guiado "Tu primera llamada a un modelo"** (Colab o JupyterLab): celdas que el participante ejecuta y completa, con celdas de autocomprobación que le dicen si lo logró. Es efectivo porque la habilidad de AE2 y AE4 es escribir y ejecutar código que consume una API y procesa texto: el notebook la ejercita directamente y termina en el código base de las actividades 1 y 2.
2. **Video interactivo "Del prompt a la respuesta"**: 6 minutos con 5 preguntas incrustadas. Es efectivo porque obliga a leer una solicitud y una respuesta reales de la documentación y a predecir el efecto de un prompt y de `temperature` antes de verlo, que son las habilidades de AE2 (2.1) y AE3.

Herramienta 1 · Notebook guiado "Tu primera llamada a un modelo"

AE: AE2 y AE4 · **Indicadores:** 2.1, 2.2, 4.1, 4.2 · **Duración:** 60 a 90 minutos

Formato: notebook publicado en el LMS con enlace para abrirlo en Colab o descargarlo para JupyterLab. Cada sección termina con una celda de autocomprobación con `assert` y un mensaje que explica qué revisar si falla.

Sección	Qué hace el participante	Celda de autocomprobación
0 · Preparar	Instala <code>httpx</code> , carga la clave desde el entorno (en Colab, desde <code>Secrets</code> con <code>userdata.get("OPENAI_API_KEY")</code> ; en local, con una variable de entorno) y define <code>MODELO_CHAT</code> y <code>MODELO_EMBEDDINGS</code> .	<code>assert os.environ.get("OPENAI_API_KEY"), "La clave no está en el entorno: revisa Secrets o tu .env" y una comprobación de que la clave no aparece escrita en ninguna celda.</code>
1 · GET	Lista los modelos con <code>httpx.get(".../models", headers=...)</code> y muestra los 5 primeros identificadores.	<code>assert r.status_code == 200, f"Estado {r.status_code}: revisa la clave o la URL"</code>
2 · POST de generación	Arma a mano el JSON de una solicitud a <code>/chat/completions</code> con un mensaje de sistema y uno de usuario; imprime la respuesta completa y marca en ella <code>choices</code> , <code>message</code> , <code>content</code> y <code>usage</code> .	<code>assert "choices" in datos and datos["choices"][0]["message"]["content"]</code>
3 · Tokens y costo	Lee <code>usage</code> y calcula cuántos tokens de entrada y de salida consumió; repite con un mensaje el doble de largo.	El participante escribe la relación que observó; la celda verifica que <code>total_tokens</code> aumentó.
4 · Temperature	Ejecuta el mismo prompt 3 veces con <code>temperature</code> 0,2 y 3 veces con 0,9; compara las respuestas.	Pregunta en el notebook: "¿con cuál variaron más?" y verifica que haya 6 respuestas guardadas.
5 · Embeddings	Obtiene embeddings de tres frases (dos parecidas y una distinta) y calcula la similitud de coseno entre ellas.	<code>assert sim(parecidas) > sim(distintas), "Las frases parecidas deberían estar más cerca"</code>
6 · Errores	Hace una solicitud con una clave inválida a propósito y lee el código y el mensaje de error.	<code>assert r.status_code == 401</code> y el participante anota qué significa 401, 429 y 500.
7 · Limpiar texto	Aplica <code>limpiar()</code> (entregada) al ticket T1 con HTML y firma; compara antes y después.	<code>assert "<p>" not in limpio and "@" not in limpio</code>
8 · Tokenizar	Tokeniza con <code>spaCy</code> , quita stopwords, obtiene lemas y bigramas del ticket limpio.	<code>assert "iniciar" in tokens</code> y verificación de que ninguna stopword quedó en la lista.
9 · Hacia la actividad 1	Copia lo que funcionó a un archivo <code>cliente_ia.py</code> con una primera versión de <code>ClienteIA</code> .	Mensaje final con el enlace a la actividad 1.

Seguridad (se explica en la sección 0): la clave nunca se escribe en una celda; un notebook compartido con una clave escrita la expone a cualquiera que lo abra.

Herramienta 2 · Video interactivo "Del prompt a la respuesta"

AE: AE2 y AE3 · **Indicadores:** 2.1, 3.1, 3.2, 3.3 · **Duración:** 6 min de video + preguntas

Formato: H5P *Interactive Video* (o equivalente). Las preguntas detienen el video; cada respuesta muestra su retroalimentación.

Producción: grabación de pantalla con la documentación oficial y un editor de código, con la voz del tutor.

Guion

Tiempo	Pantalla	Locución
0:00–0:20	El ticket T1 desordenado y su resumen ideal	"Entre este ticket y este resumen hay tres cosas: una solicitud HTTP, un prompt y unos parámetros. Vamos a verlas en orden."
0:20–1:30	La documentación oficial del endpoint de chat, con la solicitud de ejemplo resaltada	"La documentación siempre te dice tres cosas: la URL, cómo te autenticas y qué campos lleva el cuerpo. Aquí: <code>model</code> , <code>messages</code> con sus roles y parámetros opcionales como <code>temperature</code> ."
1:30	Pregunta 1	
1:30–2:30	La respuesta JSON de ejemplo	"La respuesta trae el texto dentro de <code>choices</code> , y el consumo en <code>usage</code> . Si la API responde con error, no hay <code>choices</code> : por eso primero se mira el código de estado."
2:30	Pregunta 2	
2:30–3:40	Dos prompts lado a lado: zero-shot y few-shot	"Zero-shot: solo la instrucción. Few-shot: la instrucción más ejemplos de lo que esperas. Los ejemplos enseñan el formato mejor que cualquier explicación, pero gastan tokens."
3:40	Pregunta 3	
3:40–4:50	La misma solicitud con <code>temperature</code> 0,2 y 0,9, tres veces cada una	" <code>temperature</code> controla cuánto se arriesga el modelo. Para resumir, bajo. Para lluvia de ideas, más alto. <code>max_tokens</code> corta la respuesta; <code>top_p</code> es otra forma de acotar la variedad: ajusta uno de los dos, no ambos a la vez."
4:50	Pregunta 4	
4:50–5:40	Un resumen que inventa un dato que no estaba en el ticket	"Los modelos pueden inventar con total seguridad. Por eso el prompt dice 'no agregues datos que no estén en el ticket', y por eso vas a medir."
5:40	Pregunta 5	
5:40–6:00	La tabla de iteraciones de la actividad 2	"Cada cambio de prompt, a la tabla: qué cambiaste y qué pasó. Así se decide con evidencia."

Preguntas incrustadas

- (1:30 · selección)** ¿Dónde va la clave en la solicitud?
a) En el cuerpo JSON, campo `key` · **b) En el encabezado `Authorization: Bearer <clave>`** · c) Al final de la URL
Correcta: "Así la clave no queda en registros de URL ni en el cuerpo." · *Incorrecta:* "Revisa la sección de autenticación de la documentación: va en un encabezado."
- (2:30 · verdadero o falso)** "Si la API responde 429, el texto igual está en `choices`."
Falso. *Retroalimentación:* "429 es demasiadas solicitudes: la respuesta trae un error, no `choices`. Espera y reintenta."
- (3:40 · selección)** Quieres que todos los resúmenes empiecen con "El usuario...". ¿Qué es lo más efectivo?
a) Subir `temperature` · **b) Agregar dos ejemplos (few-shot) que empiecen así** · c) Aumentar `max_tokens`
- (4:50 · selección)** Necesitas el mismo resumen cada vez que corres el proceso. ¿Qué haces?
a) Bajar `temperature` a 0–0,2 · b) Subir `top_p` a 1 · c) Quitar el mensaje de sistema
- (5:40 · respuesta corta)** Escribe una restricción que agregarías al prompt para evitar que el modelo invente datos.
Retroalimentación modelo: "Por ejemplo: 'Usa solo información que esté en el ticket; si falta, escribe «sin datos»'."

Registro: el LMS guarda las respuestas; el tutor revisa quién falló las preguntas 3 y 4 antes de la sesión en vivo del tramo 3.

PF1822 · Módulo 2 · C2 — Actividades prácticas con respuesta modelada

Estado: borrador · **Va en:** Anexo N°2, sección VI b) · **Guía:** Anexo N°7, num. 7, pág. 109

Para el 7,0: 2 actividades prácticas distintas que permitan adquirir la habilidad. Siguen el ejemplo de la guía (pág. 110): una de **resolución de problemas** apoyada en tutoriales y videos, y otra de **análisis de caso** con **gamificación y simulación**, ambas con respuesta modelada.

Aviso para quien revise: el código de las respuestas modeladas está escrito y revisado a mano, pero **no se ejecutó** en la máquina donde se produjo este kit (no tiene Python). Antes de publicarlo, el tutor lo corre una vez con `pytest` y con una clave de prueba.

	Actividad 1	Actividad 2
Nombre	Un cliente de API para el resumidor	Laboratorio de prompts y métricas
Técnica	Resolución de problemas	Análisis de caso + simulación con tablero de puntajes
AE que cubre	AE1, AE2, AE3	AE3, AE4
Indicadores	1.2, 1.3, 2.1, 2.2, 2.3, 3.1, 3.2	3.1, 3.2, 3.3, 4.1, 4.2, 4.3
Tramo del módulo	1 y 2; la parte C abre el tramo 3	3 y 4
Tiempo estimado	7 h (2 h guiadas + 5 h autónomas)	6 h (1,5 h sincrónica + 4,5 h autónomas)
Apoyos	Notebook guiado R05, cápsulas R03	Video interactivo R06
Producto	Diagrama + <code>cliente_ia.py</code> + <code>test_cliente_ia.py</code> + <code>prompts.md</code>	Notebook del pipeline + registro de prompts + tabla de métricas
Se evalúa con	Rúbrica de solución (B2-1)	Rúbrica de solución (B2-1) + bitácora (B4-d)

Actividad 1 · Un cliente de API para el resumidor

Enunciado para el participante

La mesa de ayuda de Nube Sur (empresa ficticia) quiere un resumidor de tickets. Antes de escribir prompts, el equipo necesita dos cosas: saber cómo va a estar armada la aplicación y tener una pieza de código confiable para hablar con el modelo.

Parte A — Arquitectura (AE1). Dibuja el diagrama funcional del resumidor con estos componentes: la aplicación de la mesa de ayuda (cliente), el servidor que expone la API interna, el módulo de preprocesamiento, el modelo de IA externo y una base de datos o *vector store*. Explica en una línea qué hace cada uno y responde: **¿dónde se guarda la clave de la API y por qué no puede estar en el cliente?** Justifica tu arquitectura con dos criterios: escalabilidad, seguridad, costo o mantenimiento.

Parte B — Cliente de API (AE2). Programa en Python el módulo `cliente_ia.py` con una clase `ClienteIA` que:

1. lea la clave y los nombres de modelo desde **variables de entorno**, nunca desde el código;
2. liste los modelos disponibles con una solicitud **GET**;
3. genere texto con una solicitud **POST** al endpoint de chat, con `temperature` y límite de tokens configurables, y devuelva el texto y los tokens usados;
4. obtenga **embeddings** de una lista de textos con una solicitud **POST**;
5. informe los errores HTTP con un mensaje claro;

6. esté documentada con **docstrings**.

Escribe además `test_cliente_ia.py` con **al menos tres pruebas unitarias** que se ejecuten **sin conexión**, simulando la API.

Parte C — El prompt que no sirve (AE3). El equipo probó este prompt y el jefe de soporte lo rechazó:

```
Resume esto: <ticket>
```

Con `temperature 1,0` y el ticket *"La app móvil se cierra al abrir el historial de pedidos en Android 14; en iOS funciona."* devolvió dos salidas distintas: (a) *"Hay un problema con la app. Se recomienda reinstalarla y actualizar el sistema operativo para solucionar el error de forma definitiva."* y (b) *"App falla."*

1. Diagnostica **tres fallas** del prompt o de sus parámetros: qué indicación falta y qué efecto tuvo en las salidas.
2. Reescribe el prompt como **zero-shot** con rol, tarea, formato y restricciones. Elige `temperature` y límite de tokens y justifica cada uno en una línea.
3. Pruébalo con tu `ClienteIA` sobre ese ticket y sobre este otro: *"No puedo descargar la factura de agosto desde el portal; el botón queda cargando."*

Entrega: el diagrama (imagen o Mermaid), los dos archivos `.py`, la salida de `pytest`, `prompts.md` con el diagnóstico, el prompt corregido y sus dos salidas, y un `README.md` breve. Tu clave **no** puede aparecer en ningún archivo ni captura.

Respuesta modelada

Parte A — diagrama de referencia:

DIAGRAMA (MERMAID)

```
graph LR
    U[App de la mesa de ayuda<br/>navegador] -->|ticket| S[Servidor API interno<br/>guarda la clave]
    S --> P[Preprocesamiento<br/>limpia HTML, firmas, datos personales]
    P --> M[Modelo de IA externo<br/>vía API REST]
    M -->|resumen| S
    S --> DB[(Base de datos /<br/>vector store)]
    S -->|resumen + prioridad| U
```

Componente	Función
App de la mesa de ayuda	Muestra el ticket y su resumen; nunca conoce la clave.
Servidor API interno	Recibe el ticket, orquesta el flujo y es el único que guarda la clave (variable de entorno).
Preprocesamiento	Limpia el texto y quita datos personales antes de enviarlo: menos tokens y menos riesgo.
Modelo de IA externo	Genera el resumen y los embeddings.
Base de datos / vector store	Guarda tickets, resúmenes y embeddings para buscar tickets parecidos.

Por qué la clave no va en el cliente: todo lo que llega al navegador puede leerlo cualquiera; una clave expuesta se usa para gastar a cuenta de la empresa. *Justificación modelo:* **seguridad**, porque la clave y los datos personales quedan del lado del servidor; **escalabilidad**, porque el servidor puede encolar tickets y procesarlos en lote sin cambiar la aplicación de la mesa de ayuda.

Parte B — `cliente_ia.py` de referencia:

```
"""Cliente mínimo y reutilizable para APIs de modelos generativos compatibles con OpenAI."""
from __future__ import annotations

import os

import httpx

class ErrorAPI(Exception):
```

```
"""Error devuelto por la API, con su código HTTP y el mensaje del servicio."""
```

```
def __init__(self, estado: int, mensaje: str):  
    super().__init__(f"HTTP {estado}: {mensaje}")  
    self.estado = estado  
    self.mensaje = mensaje
```

```
class ClienteIA:
```

```
    """Encapsula las llamadas de listado de modelos, generación y embeddings.
```

```
    Args:
```

```
        base_url: URL base de la API, por ejemplo "https://api.openai.com/v1".  
        clave_api: clave de acceso. Si es None, se lee de la variable OPENAI_API_KEY.  
        modelo_chat: modelo de generación. Si es None, se lee de MODELO_CHAT.  
        modelo_embeddings: modelo de embeddings. Si es None, se lee de MODELO_EMBEDDINGS.  
        timeout: segundos máximos de espera por solicitud.  
        http: cliente httpx ya construido; las pruebas lo usan para simular la API.
```

```
    """
```

```
def __init__(self, base_url: str = "https://api.openai.com/v1", clave_api: str | None = None,  
              modelo_chat: str | None = None, modelo_embeddings: str | None = None,  
              timeout: float = 30.0, http: httpx.Client | None = None):  
    self.clave_api = clave_api or os.environ.get("OPENAI_API_KEY")  
    if not self.clave_api:  
        raise ValueError("Falta OPENAI_API_KEY: defínela en el entorno, nunca en el código.")  
    self.modelo_chat = modelo_chat or os.environ.get("MODELO_CHAT")  
    self.modelo_embeddings = modelo_embeddings or os.environ.get("MODELO_EMBEDDINGS")  
    self.http = http or httpx.Client(base_url=base_url, timeout=timeout)  
    self.http.headers["Authorization"] = f"Bearer {self.clave_api}"
```

```
def _revisar(self, respuesta: httpx.Response) -> dict:  
    """Devuelve el JSON de la respuesta o lanza ErrorAPI si el estado es 400 o más."""  
    if respuesta.status_code >= 400:  
        raise ErrorAPI(respuesta.status_code, respuesta.text[:300])  
    return respuesta.json()
```

```
def listar_modelos(self) -> list[str]:  
    """Devuelve los identificadores de los modelos disponibles (GET /models)."""  
    datos = self._revisar(self.http.get("/models"))  
    return [m["id"] for m in datos.get("data", [])]
```

```
def generar(self, mensajes: list[dict], temperature: float = 0.2, max_tokens: int = 300) -> dict:  
    """Envía una conversación al endpoint de chat (POST /chat/completions).
```

```
    Args:
```

```
        mensajes: lista de {"role": "system" | "user" | "assistant", "content": str}.  
        temperature: aleatoriedad de la respuesta; baja para resúmenes consistentes.  
        max_tokens: máximo de tokens de la respuesta.
```

```
    Returns:
```

```
        {"texto": str, "tokens": dict} con el texto generado y el uso de tokens.
```

```
    """
```

```
    if not self.modelo_chat:  
        raise ValueError("Falta el modelo: define MODELO_CHAT.")  
    datos = self._revisar(self.http.post("/chat/completions", json={  
        "model": self.modelo_chat, "messages": mensajes,  
        "temperature": temperature, "max_tokens": max_tokens,  
    }))  
    return {"texto": datos["choices"][0]["message"]["content"], "tokens": datos.get("usage", {})}
```

```
def embeddings(self, textos: list[str]) -> list[list[float]]:  
    """Devuelve un vector por texto, en el mismo orden de entrada (POST /embeddings)."""  
    if not self.modelo_embeddings:  
        raise ValueError("Falta el modelo: define MODELO_EMBEDDINGS.")  
    datos = self._revisar(self.http.post("/embeddings", json={  
        "model": self.modelo_embeddings, "input": textos,
```

```

    )))
    return [d["embedding"] for d in sorted(datos["data"], key=lambda d: d["index"])]

```

test_cliente_ia.py de referencia (sin conexión, con `httpx.MockTransport`):

```

import httpx
import pytest

from cliente_ia import ClienteIA, ErrorAPI

def cliente_simulado(cuerpo: dict, estado: int = 200, registro: list | None = None) -> ClienteIA:
    """Crea un ClienteIA cuyas solicitudes responde una API falsa."""
    def responder(solicitud: httpx.Request) -> httpx.Response:
        if registro is not None:
            registro.append(solicitud)
        return httpx.Response(estado, json=cuerpo)

    http = httpx.Client(base_url="https://api.test/v1", transport=httpx.MockTransport(responder))
    return ClienteIA(clave_api="clave-de-prueba", modelo_chat="modelo-x",
                    modelo_embeddings="embeddings-x", http=http)

def test_generar_devuelve_texto_tokens_y_autentica():
    registro = []
    cliente = cliente_simulado({"choices": [{"message": {"content": "Resumen"}}],
                               "usage": {"total_tokens": 42}}, registro=registro)
    respuesta = cliente.generar([{"role": "user", "content": "hola"}])
    assert respuesta == {"texto": "Resumen", "tokens": {"total_tokens": 42}}
    assert registro[0].method == "POST"
    assert registro[0].url.path == "/v1/chat/completions"
    assert registro[0].headers["Authorization"] == "Bearer clave-de-prueba"

def test_embeddings_respeto_el_orden_de_entrada():
    cliente = cliente_simulado({"data": [{"index": 1, "embedding": [0.3]},
                                         {"index": 0, "embedding": [0.1]}]})
    assert cliente.embeddings(["a", "b"]) == [[0.1], [0.3]]

def test_listar_modelos_usa_get():
    registro = []
    cliente = cliente_simulado({"data": [{"id": "m1"}, {"id": "m2"}]}, registro=registro)
    assert cliente.listar_modelos() == ["m1", "m2"]
    assert registro[0].method == "GET"

def test_error_http_se_informa_con_su_codigo():
    cliente = cliente_simulado({"error": {"message": "clave inválida"}}, estado=401)
    with pytest.raises(ErrorAPI) as error:
        cliente.generar([{"role": "user", "content": "hola"}])
    assert error.value.estado == 401

def test_sin_clave_no_arranca(monkeypatch):
    monkeypatch.delenv("OPENAI_API_KEY", raising=False)
    with pytest.raises(ValueError):
        ClienteIA()

```

Salida esperada: 5 passed.

Uso con Hugging Face. La misma clase sirve con cualquier API compatible con el formato de OpenAI cambiando `base_url`, la clave y el modelo. **PENDIENTE:** el tutor confirma en la documentación vigente de Hugging Face (Inference Providers) la URL base compatible y un modelo de ejemplo antes de publicar la actividad; las URL de ese servicio han cambiado.

Nota sobre parámetros. Algunos modelos recientes piden `max_completion_tokens` en vez de `max_tokens` o no aceptan `temperature`. El participante lo comprueba en la documentación del modelo que use: es parte del indicador 2.1.

Parte C — respuesta modelada.

Diagnóstico:

1. **No dice qué es un buen resumen** (largo, foco, persona gramatical). Efecto: la salida (b) es tan corta que no nombra la plataforma ni la condición en que falla.
2. **No prohíbe agregar información.** Efecto: la salida (a) inventa una solución ("reinstalarla") que no está en el ticket.
3. **temperature 1,0 y sin límite de tokens** para una tarea que debe ser estable. Efecto: dos ejecuciones dan resultados muy distintos y el largo no está acotado.

Prompt corregido (zero-shot):

```
[system] Eres analista de soporte de Nube Sur. Resume el ticket en UNA oración de máximo 25 palabras, en tercera persona. Indica qué falla, en qué producto y en qué condición ocurre. No propongas soluciones ni agregues datos que no estén en el ticket.  
[user] Ticket: <ticket limpio>
```

Parámetros: `temperature 0,2`, porque resumir es extraer y se busca la misma respuesta cada vez; límite de 60 tokens, porque una oración de 25 palabras cabe con margen.

Salidas esperadas (pueden variar en la redacción): "La app móvil se cierra al abrir el historial de pedidos en Android 14; en iOS funciona correctamente." y "El usuario no puede descargar la factura de agosto desde el portal porque el botón queda cargando."

Nivel logrado (indicadores 3.1 y 3.2): identifica al menos dos fallas con su efecto; el prompt corregido tiene rol, tarea, formato y restricción; los parámetros se justifican.

Errores típicos: la clave escrita en el código o en el notebook (falla grave, criterio 2 de la rúbrica); pruebas que llaman a la API real y fallan sin internet; no revisar el código de estado y leer `choices` de una respuesta de error. En la parte C: corregir el prompt agregando la solución correcta en vez de prohibir las soluciones.

Actividad 2 · Laboratorio de prompts y métricas

Enunciado para el participante

El jefe de soporte de Nube Sur va a elegir **una** configuración para el resumidor y quiere evidencia, no opiniones. Te entrega 5 tickets reales (anonimizados, ficticios para el curso) con el resumen que escribió una persona de su equipo para cada uno. Tu misión es simular la decisión: preparar los tickets, probar estrategias de prompt y medir cuál resume mejor.

1. **Preprocesa (AE4).** Programa funciones que limpien HTML y firmas, reemplacen correos y enlaces por marcas, normalicen espacios y tokenicen. Explica con un ticket qué cambió.
2. **Diseña (AE3).** Escribe un prompt **zero-shot** y uno **few-shot** (con dos ejemplos) para resumir un ticket en una oración de máximo 25 palabras.
3. **Itera (AE3).** Registra al menos tres versiones del prompt con qué cambiaste y qué efecto tuvo. Prueba cada estrategia con `temperature 0,2` y `0,9`.
4. **Mide (AE4).** Calcula ROUGE-1, ROUGE-L y BLEU de cada resumen contra la referencia, valora coherencia y relevancia de 1 a 3, y registra todo en `resultados.csv`.
5. **Decide.** Recomienda una configuración en 5 líneas, con números.

Tablero del laboratorio (gamificación): cada participante publica en el LMS su mejor ROUGE-L promedio y la configuración que lo logró. Hay insignias para **"Mejor ROUGE-L"**, **"Prompt más corto con ROUGE-L sobre 0,5"** y **"Mejor bitácora de iteraciones"** (esta la elige el tutor). Lo que puntúa en la nota no es el primer lugar, sino el proceso registrado.

Insumos que entrega el LMS

id	Ticket (tal como llega)	Resumen de referencia
T1	<p>Hola equipo,</p><p>Desde ayer en la tarde no puedo iniciar sesión en la app de facturación después de cambiar mi contraseña. Me dice "credenciales inválidas" aunque estoy segura de que la escribo bien.</p><p>Saludos,
Marta</p> + firma -- Marta Vidal · Contabilidad · marta.vidal@cliente.test	El usuario no puede iniciar sesión en la app de facturación tras cambiar su contraseña.
T2	Buenas, el reporte mensual de ventas en PDF sale con las tildes rotas (aparece "Compañ±Ãa"). Pasa desde la actualización del lunes. Enviado desde mi teléfono	El reporte mensual de ventas en PDF muestra caracteres rotos desde la actualización del lunes.
T3	URGENTE!!! la API de pagos responde error 504 en el 30% de las llamadas desde las 09:00, afecta a clientes en Chile y Perú. Logs: https://logs.nubesur.test/abc	La API de pagos devuelve error 504 en parte de las llamadas desde las 09:00 en Chile y Perú.
T4	Hola, ¿se puede exportar el listado de clientes a Excel? No encuentro la opción en el panel. Gracias	El usuario pregunta cómo exportar el listado de clientes a Excel desde el panel.
T5	El botón "Guardar" del formulario de proveedores no hace nada en Firefox; en Chrome funciona bien. Probé borrando caché.	El botón Guardar del formulario de proveedores no funciona en Firefox.

Todos los nombres, correos y dominios son ficticios (el dominio `.test` está reservado).

Respuesta modelada

1 · Preprocesamiento (`preprocesar.py`):

```
"""Limpieza, normalización y tokenización de tickets antes de enviarlos al modelo."""
import html
import re
import unicodedata

import spacy

nlp = spacy.load("es_core_news_sm") # python -m spacy download es_core_news_sm

def limpiar(texto: str) -> str:
    """Quita HTML, firma y datos personales; deja un texto listo para el modelo."""
    texto = html.unescape(texto)
    texto = re.sub(r"<br\s*/?>|</p>", "\n", texto, flags=re.IGNORECASE)
    texto = re.sub(r"<[^>]+>", " ", texto) # resto de etiquetas HTML
    texto = re.split(r"\n\s*--\s", texto)[0] # todo lo que sigue a la firma "-- "
    texto = re.sub(r"Enviado desde mi \w+", " ", texto, flags=re.IGNORECASE)
    texto = re.sub(r"\S+@\S+", "[correo]", texto) # datos personales fuera
    texto = re.sub(r"https?://\S+", "[enlace]", texto)
    texto = re.sub(r"!{2,}", "!", texto)
    return re.sub(r"\s+", " ", texto).strip()

def quitar_tildes(texto: str) -> str:
    """Pasa 'sesión' a 'sesion'; se usa solo para comparar textos al medir."""
    return "".join(c for c in unicodedata.normalize("NFD", texto) if unicodedata.category(c) != "Mn")

def tokenizar(texto: str, lematizar: bool = False, sin_stopwords: bool = False) -> list[str]:
    """Tokeniza con spaCy; opcionalmente lematiza y quita stopwords y puntuación."""
    tokens = []
    for t in nlp(texto.lower()):
        if t.is_punct or t.is_space or (sin_stopwords and t.is_stop):
            continue
        tokens.append(t.lemma_ if lematizar else t.text)
```

```
return tokens
```

```
def ngramas(tokens: list[str], n: int = 2) -> list[tuple[str, ...]]:
    """Devuelve los n-gramas consecutivos de una lista de tokens."""
    return list(zip(*(tokens[i:] for i in range(n))))
```

Ejemplo con T1: limpiar deja "Hola equipo, Desde ayer en la tarde no puedo iniciar sesión en la app de facturación después de cambiar mi contraseña. Me dice "credenciales inválidas" aunque estoy segura de que la escribo bien. Saludos, Marta": se van las etiquetas y la firma con el correo. tokenizar(..., sin_stopwords=True) deja, entre otros, iniciar, sesión, app, facturación, contraseña. El problema de T2 ("Compañía") se conserva a propósito: es parte del problema que reporta el cliente.

2 • Prompts de referencia:

Zero-shot:

```
[system] Eres analista de soporte de Nube Sur. Resume el ticket en UNA oración de máximo
25 palabras, en español neutro, en tercera persona. Indica qué falla y en qué producto.
No agregues soluciones ni datos que no estén en el ticket.
[user] Ticket: <ticket limpio>
```

Few-shot: el mismo mensaje de sistema, seguido de dos ejemplos como turnos previos (user con un ticket y assistant con su resumen) antes del ticket a resumir. Ejemplos distintos de los cinco tickets evaluados, para no contaminar la medición:

```
[user] Ticket: No me llegan los correos de recuperación de clave desde el viernes.
[assistant] El usuario no recibe los correos de recuperación de clave desde el viernes.
[user] Ticket: ¿Cómo agrego un segundo administrador a mi cuenta?
[assistant] El usuario pregunta cómo agregar un segundo administrador a su cuenta.
[user] Ticket: <ticket limpio>
```

3 • Registro de iteraciones (ejemplo de nivel logrado):

Versión	Cambio	Efecto observado
v1	"Resume esto: <ticket>"	Resúmenes de 3 a 5 oraciones, a veces con soluciones inventadas. Falta control .
v2	Rol de analista + "una oración, máximo 25 palabras" + "no agregues soluciones"	Largo correcto; T4 sale como instrucción ("Exporta el listado...") en vez de pregunta. Mejora la estructura .
v3	"En tercera persona, indica qué falla y en qué producto" + few-shot con una pregunta de ejemplo	T4 queda "El usuario pregunta cómo...". Formato uniforme en los cinco. Mejora la claridad .
v3 · T=0,9	Misma v3 con temperature 0,9	Resúmenes más variados entre corridas y dos con palabras que no están en el ticket. Para esta tarea conviene T=0,2 .

4 • Medición (evaluar.py):

```
"""Mide resúmenes generados contra referencias y registra los resultados."""
import csv
import re

from nltk.translate.bleu_score import SmoothingFunction, sentence_bleu
from rouge_score import rouge_scorer

from preprocesar import quitar_tildes

class TokenizadorES:
    """rouge-score descarta por defecto las letras con tilde; este tokenizador las conserva."""

    def tokenize(self, texto: str) -> list[str]:
```

```

return re.findall(r"\w+", quitar_tildes(texto.lower()))

tok = TokenizadorES()
rouge = rouge_scorer.RougeScorer(["rouge1", "rougeL"], tokenizer=tok)
suavizado = SmoothingFunction().method1 # textos cortos: sin suavizado BLEU da 0 con facilidad

def medir(referencia: str, generado: str) -> dict:
    """Devuelve ROUGE-1, ROUGE-L (F1) y BLEU de un resumen frente a su referencia."""
    r = rouge.score(referencia, generado)
    bleu = sentence_bleu([tok.tokenize(referencia)], tok.tokenize(generado), smoothing_function=suavizado)
    return {"rouge1_f": round(r["rouge1"].fmeasure, 3), "rougeL_f": round(r["rougeL"].fmeasure, 3),
            "bleu": round(bleu, 3)}

def registrar(filas: list[dict], ruta: str = "resultados.csv") -> None:
    """Escribe el protocolo: una fila por ticket, estrategia y temperature."""
    campos = ["ticket", "estrategia", "version", "temperature", "rouge1_f", "rougeL_f", "bleu",
              "tokens", "coherencia_1a3", "relevancia_1a3", "comentario"]
    with open(ruta, "w", newline="", encoding="utf-8") as f:
        escritor = csv.DictWriter(f, fieldnames=campos)
        escritor.writeheader()
        escritor.writerows(filas)

```

Cálculo de comprobación (para verificar que la métrica está bien armada). Referencia de T1: "El usuario no puede iniciar sesión en la app de facturación tras cambiar su contraseña." Resumen generado de ejemplo: "Usuario no logra iniciar sesión en la app de facturación después de cambiar la contraseña." Ambos tienen 15 palabras y comparten 11 (usuario, no, iniciar, sesión, en, la, app, de, facturación, cambiar, contraseña), que además aparecen en el mismo orden. Por lo tanto **ROUGE-1 F1 = ROUGE-L F1 = 11/15 = 0,733**. Si el participante obtiene un valor muy distinto con este par, su tokenizador está perdiendo las palabras con tilde.

Protocolo de coherencia y relevancia (valoración humana, 1 a 3): coherencia 3 = una oración gramatical y sin contradicciones; relevancia 3 = nombra la falla y el producto del ticket sin agregar nada que no esté en él. Las valora el participante y el tutor revisa una muestra.

5 • Recomendación modelo:

Recomiendo few-shot v3 con temperature 0,2: obtuvo el ROUGE-L promedio más alto de las cuatro configuraciones, todos sus resúmenes tuvieron coherencia y relevancia 3, y fue la más estable entre corridas. El zero-shot quedó cerca en ROUGE pero falló el formato en T4. Con temperature 0,9 bajó la relevancia porque apareció información que no estaba en el ticket. Costo: el few-shot usa más tokens de entrada por los ejemplos; con 300 tickets diarios conviene medir si esa diferencia importa.

(Los números de cada participante varían según el modelo y la corrida; lo que se evalúa es que la recomendación se apoye en su propia tabla.)

Qué distingue una respuesta débil: elige la configuración "que se ve mejor" sin tabla; usa los mismos tickets de evaluación como ejemplos del few-shot; o reporta ROUGE calculado con el tokenizador por defecto, que en español descarta las letras con tilde.

PF1822 · Módulo 2 · B1 — Indicadores de logro

Estado: borrador · **Va en:** Anexo N°2, sección V.1 · **Guía:** Anexo N°7, num. 2

Para el 7,º: 3 indicadores por aprendizaje esperado → 12.

Fórmula de cada indicador: **verbo en presente + contenido + condición**. Los aprendizajes esperados van TEXTUALES de SIPFOR. No se reformulan.

Aprendizaje esperado (textual del plan)	Indicadores de logro
AE1. ANALIZAR EL ROL DE LOS MODELOS GENERATIVOS EN EL DESARROLLO DE APLICACIONES, DESCRIBIENDO SUS TIPOS, CASOS DE USO, TECNOLOGÍAS ASOCIADAS Y REPRESENTANDO ARQUITECTURAS FUNCIONALES QUE INTEGREN ESTOS MODELOS MEDIANTE APIS.	1.1 Clasifica modelos generativos por tipo de salida y por familia técnica (autorregresivos, de difusión, basados en transformers) en un cuadro que asocia a cada uno un caso de uso organizacional. 1.2 Representa la arquitectura funcional de una aplicación con IA generativa en un diagrama con cliente, servidor API, preprocesamiento, modelo y almacenamiento, explicando la función de cada componente. 1.3 Justifica la elección de una arquitectura para un caso dado con al menos dos criterios entre escalabilidad, seguridad, costo y mantenimiento.
AE2. UTILIZAR UN MODELO DE IA A TRAVÉS DE API REST UTILIZANDO SOLICITUDES HTTP Y AUTENTICACIÓN, BASÁNDOSE EN LA DOCUMENTACIÓN OFICIAL DE OPENAI Y HUGGING FACE.	2.1 Explica la estructura de una solicitud y una respuesta de los endpoints de generación y de embeddings, citando la sección de la documentación oficial donde está cada campo. 2.2 Implementa solicitudes HTTP GET y POST autenticadas con variables de entorno, obteniendo respuestas válidas de los endpoints de generación y de embeddings. 2.3 Encapsula el consumo de la API en una clase de Python reutilizable, documentada con docstrings y verificada con al menos tres pruebas unitarias que se ejecutan sin conexión.
AE3. FORMULAR PROMPTS EFECTIVOS PARA MODELOS GENERATIVOS EN ESCENARIOS ZERO-SHOT O FEW-SHOT, CONSIDERANDO LA TAREA SOLICITADA Y LOS PARÁMETROS DE GENERACIÓN DISPONIBLES.	3.1 Explica la diferencia entre prompting zero-shot y few-shot con un ejemplo propio de cada uno, indicando una ventaja y una limitación de cada estrategia. 3.2 Elabora prompts con rol, contexto, instrucciones, restricciones y formato de salida para tareas de resumen, generación de código y reformulación, eligiendo la estrategia según el objetivo. 3.3 Ajusta un prompt en al menos tres iteraciones registradas, describiendo cada cambio y su efecto en la claridad, la estructura o el control de la respuesta, incluido el efecto de temperature.
AE4. IMPLEMENTAR UN PIPELINE DE PREPROCESAMIENTO Y EVALUACIÓN BÁSICA DE TEXTO EN PYTHON, APLICANDO TÉCNICAS DE LIMPIEZA, NORMALIZACIÓN Y MÉTRICAS INICIALES DE CALIDAD, DE ACUERDO CON LOS REQUISITOS DE ENTRADA Y SALIDA DEL MODELO.	4.1 Explica los conceptos de token, lema, stopword y n-grama, y los criterios de coherencia y relevancia, aplicándolos a un texto del caso. 4.2 Codifica en Python funciones reutilizables de limpieza, normalización y tokenización que procesan un corpus de documentos sin errores y devuelven textos listos para el modelo. 4.3 Calcula ROUGE y BLEU de respuestas generadas frente a resúmenes de referencia y registra los resultados, junto con la valoración de coherencia y relevancia, en una tabla con protocolo definido.

Trazabilidad con los criterios del plan

Criterio del plan (SIPFOR)	Indicador
1.1 clasifica tipos de modelos generativos	1.1
1.2 representa un diagrama funcional	1.2
1.3 analiza la elección de arquitectura	1.3
2.1 explica la estructura de los endpoints de generación y embeddings	2.1
2.2 implementa llamadas GET y POST	2.2
2.3 función o clase modular con docstrings y pruebas	2.3
3.1 explica la función del prompt y zero/few-shot	3.1
3.2 elabora prompts para tareas específicas	3.2
3.3 ajusta un prompt de forma iterativa	3.3
4.1 explica conceptos de NLP y evaluación	4.1
4.2 codifica limpieza, tokenización y normalización	4.2
4.3 calcula métricas y registra resultados	4.3

PF1822 · Módulo 2 · B2 — Instrumentos de evaluación

Estado: borrador · **Va en:** Anexo N°2, sección V.2 · **Guía:** Anexo N°7, num. 3, 3.1–3.3

Para el 7,0: 3 instrumentos distintos, cada uno con todos los aprendizajes esperados.

- Rúbrica de revisión de la solución** · familia observación (3.1) · evalúa las actividades 1 y 2
- Proyecto "Asistente de preguntas frecuentes para Nube Sur"** · familia desempeño (3.2), proyecto individual · evaluación sumativa del módulo
- Prueba de conceptos y lectura de código** · familia objetiva (3.3), selección múltiple y respuesta breve

Tabla de especificaciones

Instrumento	AE1	AE2	AE3	AE4	Momento	Ponderación sugerida
1 · Rúbrica de solución	criterio 1	criterios 2 y 3	criterio 4	criterios 5 y 6	Actividades 1 y 2	35 %
2 · Proyecto FAQ	req. 1	req. 2	req. 3	req. 4	Cierre del módulo	45 %
3 · Prueba objetiva	ítems 1–3	ítems 4–6	ítems 7–9	ítems 10–12	Cierre del tramo 4	20 %

Escala de notas (exigencia 60 %, escala chilena de 1,0 a 7,0), para los tres: si el puntaje p es mayor o igual a $0,6 \times P$, $\text{nota} = 4,0 + 3 \times (p - 0,6 \times P) / (0,4 \times P)$; si es menor, $\text{nota} = 1,0 + 3 \times p / (0,6 \times P)$. Se redondea a un decimal.

Instrumento 1 · Rúbrica de revisión de la solución (observación)

El tutor la aplica con el repositorio del participante abierto y ejecutando sus pruebas.

Criterio (AE · indicadores)	4 · Logrado	3 · Mayormente logrado	2 · Parcialmente logrado	1 · No logrado
1. Arquitectura (AE1 · 1.1, 1.2, 1.3)	Diagrama con los cinco componentes, función de cada uno y justificación con dos criterios; la clave queda del lado del servidor	Diagrama completo con justificación de un solo criterio	Faltan componentes o no explica dónde va la clave	Sin diagrama o no corresponde a una app con IA
2. Consumo de la API (AE2 · 2.1, 2.2)	GET y POST autenticados con variables de entorno; maneja errores HTTP; explica los campos de solicitud y respuesta con la documentación oficial	Llamadas correctas sin manejo de errores o sin referencia a la documentación	Llamadas que funcionan solo con la clave escrita en el código	No logra una respuesta válida
3. Modularidad y pruebas (AE2 · 2.3)	Clase reutilizable con docstrings y 3 o más pruebas que pasan sin conexión	Clase documentada con pruebas que requieren conexión	Funciones sueltas o sin docstrings	Sin estructura reutilizable ni pruebas
4. Prompts (AE3 · 3.1, 3.2, 3.3)	Zero-shot y few-shot bien contruidos (rol, contexto, restricciones, formato); 3 o más iteraciones registradas con su efecto; compara temperature	Ambas estrategias con menos de 3 iteraciones o sin comparar temperature	Una sola estrategia o prompts sin restricciones de formato	Prompts sin estructura ni registro
5. Preprocesamiento (AE4 · 4.1, 4.2)	Funciones reutilizables que limpian HTML, firmas y datos personales, normalizan y tokenizan el corpus sin errores; explica token, lema, stopword y n-grama	Pipeline correcto con una etapa faltante o sin explicar los conceptos	Limpieza parcial que deja ruido o datos personales	Sin preprocesamiento

Criterio (AE · indicadores)	4 · Logrado	3 · Mayormente logrado	2 · Parcialmente logrado	1 · No logrado
6. Evaluación y registro (AE4 · 4.3)	ROUGE y BLEU bien calculados (tokenizador que respeta tildes), coherencia y relevancia valoradas, CSV completo y recomendación apoyada en números	Métricas correctas con registro incompleto	Métricas mal calculadas o sin registro	Sin evaluación

Puntaje máximo: 24. En la actividad 1 se aplican los criterios 1 a 3; en la actividad 2, los criterios 4 a 6. La devolución sigue el formato de `B4-retroalimentacion.md`, producto a.

Instrumento 2 · Proyecto "Asistente de preguntas frecuentes para Nube Sur" (desempeño)

Modalidad: individual, asincrónica, 4 horas de trabajo estimado, entrega como repositorio en GitHub (o equivalente) con `README.md`.

Enunciado

La mesa de ayuda de Nube Sur recibe muchas preguntas que ya están respondidas en su lista de preguntas frecuentes (se entregan 8 pares pregunta–respuesta ficticios). Construye un asistente mínimo que, dada una pregunta de un cliente, encuentre la pregunta frecuente más parecida y redacte una respuesta breve basada **solo** en esa respuesta oficial.

- Arquitectura (AE1).** En el README: diagrama de la solución (cliente, servidor, preprocesamiento, modelo, almacén de embeddings), función de cada componente y por qué elegiste esa arquitectura (dos criterios).
- API (AE2).** Reutiliza tu `ClienteIA`: embeddings de las 8 preguntas frecuentes y de la pregunta del cliente; generación de la respuesta. Clave por variable de entorno. Al menos tres pruebas unitarias sin conexión.
- Prompts (AE3).** Un prompt few-shot que obligue a responder solo con la información de la respuesta oficial y a decir "Te derivo con un ejecutivo" cuando no alcance. Compáralo con un zero-shot y registra dos iteraciones.
- Preprocesamiento y evaluación (AE4).** Limpia y normaliza las preguntas antes de calcular embeddings. Para 5 preguntas de prueba con respuesta de referencia, calcula ROUGE-L y BLEU, valora coherencia y relevancia, y registra todo en un CSV.

Insumos: preguntas frecuentes de Nube Sur (ficticias)

#	Pregunta frecuente	Respuesta oficial
F1	¿Cómo recupero mi contraseña?	En la pantalla de ingreso, elige "Olvidé mi contraseña"; te llegará un enlace válido por 30 minutos.
F2	¿Cómo agrego un usuario a mi cuenta?	Un administrador entra a Configuración → Usuarios → Invitar y escribe el correo de la persona.
F3	¿Puedo exportar mis clientes a Excel?	Sí: en Clientes, usa el botón Exportar y elige formato XLSX.
F4	¿Qué navegadores son compatibles?	Las dos últimas versiones de Chrome, Firefox, Edge y Safari.
F5	¿Cómo cambio el plan contratado?	En Facturación → Plan puedes subir de plan en el momento; para bajar, el cambio aplica al mes siguiente.
F6	¿Dónde descargo mis facturas?	En Facturación → Documentos están todas las facturas en PDF.
F7	¿La plataforma tiene API?	Sí, con clave por cuenta; la documentación está en el menú Desarrolladores.

#	Pregunta frecuente	Respuesta oficial
F8	¿Cómo elimino mi cuenta?	Escribe a soporte desde el correo del administrador; la eliminación se confirma en 5 días hábiles.

Preguntas de prueba con respuesta de referencia (para la evaluación del requisito 4):

#	Pregunta del cliente	FAQ esperada	Respuesta de referencia
Q1	olvidé la clave y no puedo entrar!!	F1	Elige "Olvidé mi contraseña" en la pantalla de ingreso y recibirás un enlace válido por 30 minutos.
Q2	Necesito pasar la lista de clientes a una planilla	F3	En Clientes, usa Exportar y elige el formato XLSX.
Q3	¿funciona en safari?	F4	Sí, funciona en las dos últimas versiones de Safari, además de Chrome, Firefox y Edge.
Q4	quiero sumar a mi colega como usuario	F2	Un administrador puede invitarlo desde Configuración → Usuarios → Invitar con su correo.
Q5	¿cuánto cuesta el plan anual?	ninguna	Te derivo con un ejecutivo.

La Q5 no tiene respuesta en las preguntas frecuentes: el asistente debe derivar y no inventar un precio. Es la prueba del prompt del requisito 3.

Pauta de corrección (30 puntos)

Req.	AE	Qué se revisa	Puntos
1	AE1	Diagrama con los cinco componentes y su función	2
1	AE1	Justificación con dos criterios; la clave no está en el cliente	2
2	AE2	Embeddings de preguntas frecuentes y consulta; selección por similitud de coseno	3
2	AE2	Generación de respuesta con <code>ClienteIA</code> ; clave por variable de entorno	3
2	AE2	Tres o más pruebas unitarias que pasan sin conexión	3
3	AE3	Prompt few-shot con restricción de fuente y frase de derivación	3
3	AE3	Comparación con zero-shot y dos iteraciones registradas con su efecto	3
4	AE4	Limpieza y normalización antes de los embeddings	3
4	AE4	ROUGE-L y BLEU bien calculados para las 5 preguntas	3
4	AE4	Coherencia y relevancia valoradas y CSV completo	2
—	—	README claro: cómo se instala, cómo se ejecuta, qué falta	3
Total			30

Respuesta modelada (núcleo del cálculo de similitud):

<pre>import math def coseno(a: list[float], b: list[float]) -> float: """Similitud de coseno entre dos vectores: 1 = misma dirección, 0 = sin relación.""" producto = sum(x * y for x, y in zip(a, b)) return producto / (math.sqrt(sum(x * x for x in a)) * math.sqrt(sum(y * y for y in b))) def pregunta_mas_parecida(consulta: str, preguntas: list[str], cliente) -> tuple[int, float]: """Devuelve el índice de la pregunta frecuente más parecida y su similitud."""</pre>

```

vectores = cliente.embeddings([limpiar(consulta)] + [limpiar(p) for p in preguntas])
similitudes = [coseno(vectores[0], v) for v in vectores[1:]]
mejor = max(range(len(similitudes)), key=similitudes.__getitem__)
return mejor, similitudes[mejor]

```

(Con limpiar de preprocesar.py de la actividad 2.) Si la similitud máxima es baja, el asistente deriva sin llamar al modelo de generación: ahorra tokens y evita inventar.

Instrumento 3 · Prueba de conceptos y lectura de código (objetiva)

Modalidad: cuestionario del LMS, 25 minutos. 9 ítems de selección múltiple (1 punto) y 3 de respuesta breve (2 puntos).

Puntaje máximo: 15.

AE1

1. Un modelo que genera imágenes partiendo de ruido y refinándolo paso a paso es:

- a) Autorregresivo · **b) De difusión** · c) Basado en reglas · d) De agrupamiento

Respuesta: b.

2. En una aplicación con IA generativa, ¿qué componente debe guardar la clave de la API?

- a) La aplicación en el navegador · **b) El servidor o backend** · c) El vector store · d) El prompt

Respuesta: b. Lo que llega al navegador lo puede leer cualquiera.

3. (Respuesta breve, 2 puntos) Da dos razones para preprocesar un texto antes de enviarlo al modelo.

Respuesta, 1 punto cada una: menos tokens y por lo tanto menor costo; mejor calidad de respuesta al quitar ruido; protección de datos personales al no enviarlos a un tercero.

AE2

1. Para autenticarse en la API de OpenAI se envía:

- a) La clave en la URL · **b) El encabezado Authorization: Bearer <clave>** · c) Usuario y contraseña en el cuerpo · d) Una cookie de sesión

Respuesta: b.

2. Un endpoint de embeddings devuelve:

- a) Texto generado · **b) Un vector de números que representa el significado del texto** · c) Una imagen · d)

Solo la cantidad de tokens

Respuesta: b.

3. (Respuesta breve, 2 puntos) La respuesta del endpoint de chat se guardó en el diccionario `datos`. Escribe la expresión de Python que obtiene el texto generado y explica por qué conviene revisar antes el código de estado.

Respuesta: `datos["choices"][0]["message"]["content"]` (1 punto); si la API devolvió un error, la respuesta no trae `choices` y el programa falla con un `KeyError` en vez de informar el error real (1 punto).

AE3

1. Un prompt *few-shot* es el que:

- a) Se envía varias veces · **b) Incluye ejemplos de entrada y salida esperada antes de la tarea** · c) Usa temperature alta · d) No tiene instrucciones

Respuesta: b.

2. Para resúmenes técnicos consistentes entre una ejecución y otra conviene:

- a) Temperature alta (por ejemplo 1,5) · **b) Temperature baja (o a 0,3)** · c) Quitar el mensaje de sistema · d) Subir mucho `max_tokens`

Respuesta: b.

3. (Respuesta breve, 2 puntos) Mejora este prompt: "Resume esto." Escribe tu versión.

Respuesta: 2 puntos si incluye al menos tres de: rol, tarea precisa, restricción de largo, formato de salida, prohibición de agregar información; 1 punto si incluye dos.

AE4

1. Las *stopwords* son:

- a) **Palabras muy frecuentes y con poco contenido propio, como "el", "de" o "y"** · b) Palabras mal escritas ·
- c) Los signos de puntuación · d) Las palabras que el modelo no conoce

Respuesta: a.

2. ¿Qué diferencia a la lematización del stemming?

- a) Son lo mismo · **b) La lematización lleva la palabra a su forma de diccionario ("corriendo" → "correr"); el stemming solo recorta la raíz** · c) El stemming usa un diccionario y la lematización no · d) La lematización quita tildes

Respuesta: b.

3. Un resumen obtiene ROUGE-1 *recall* = 0,625 frente a su referencia. Significa que:

- a) El 62,5 % del resumen es incorrecto · **b) El 62,5 % de las palabras de la referencia aparecen en el resumen generado** · c) El resumen tiene 62,5 % de coherencia · d) El modelo acertó 62,5 de cada 100 tokens

Respuesta: b.

PF1822 · Módulo 2 · B3 — Portafolio de proyectos

Estado: borrador · **Va en:** Anexo N°2, sección V (con **enlace** y rúbrica adjunta)

Guía: Anexo N°7, num. 4.3, pág. 103 · **Para el 7,0:** 6 de 6 elementos, desarrollados.

La sección V pide el enlace al portafolio, no su descripción. Este archivo es la base común: cada institución lo publica con su identidad (ver parte B del manual).

Nombre del portafolio del participante: *Resumidor de tickets para Nube Sur.*

- 1. Guía o índice de elementos, con tipo de trabajo y estrategia didáctica
- 2. Introducción: intenciones, objetivos, punto de partida
- 3. Temas centrales con evidencias por aprendizaje esperado
- 4. Apartado de cierre como síntesis
- 5. Plataforma digital de publicación definida
- 6. Instrumento evaluativo asociado, desarrollado

(Las marcas indican que el elemento está desarrollado en este archivo; la revisión humana se marca en `01-entregables.md`.)

Elemento 1 · Guía o índice

Sección	Tipo de trabajo	Evidencia	Estrategia didáctica	AE
Introducción	Reflexión inicial	Punto de partida + autodiagnóstico	Metacognición	—
1. La arquitectura	Análisis y diseño	Cuadro de tipos de modelos + diagrama del resumidor con su justificación	Análisis de caso	AE1
2. El cliente de API	Producto técnico	<code>cliente_ia.py</code> , pruebas y salida de <code>pytest</code>	Resolución de problemas	AE2
3. Los prompts	Experimentación	Prompts zero-shot y few-shot + registro de iteraciones	Laboratorio	AE3
4. Limpiar y medir	Producto técnico + análisis	Notebook del pipeline + <code>resultados.csv</code> + recomendación	Simulación con tablero	AE4
Cierre	Síntesis	Reflexión final + plan de mejora	Metacognición	—

Elemento 2 · Introducción (plantilla que el participante completa)

Intención. Qué quiero lograr en este módulo y cómo lo voy a usar en mi trabajo como desarrollador o desarrolladora. **Objetivos.** Los cuatro aprendizajes del módulo en mis palabras, y uno que me propongo profundizar. **Punto de partida.** Mi resultado en el autodiagnóstico y una aplicación de mi entorno donde un modelo generativo podría ayudar.

Autodiagnóstico inicial (escala 1 a 4, de "no lo sé hacer" a "lo hago solo"):

1. Distingo tipos de modelos generativos y sus usos. 2. Hago una solicitud HTTP con

autenticación desde Python. 3. Escribo pruebas unitarias para mi código. 4. Diseño un prompt con ejemplos. 5. Limpio y tokenizo texto en Python. 6. Interpreto una métrica de calidad de texto. Se repite en el cierre para comparar.

Elemento 3 · Temas centrales con evidencias por aprendizaje esperado

Cada sección lleva la evidencia, una explicación de 5 a 10 líneas de qué hizo y por qué, y el indicador que demuestra.

AE (textual)	Evidencia mínima	Indicadores
AE1. ANALIZAR EL ROL DE LOS MODELOS GENERATIVOS EN EL DESARROLLO DE APLICACIONES, DESCRIBIENDO SUS TIPOS, CASOS DE USO, TECNOLOGÍAS ASOCIADAS Y REPRESENTANDO ARQUITECTURAS FUNCIONALES QUE INTEGREN ESTOS MODELOS MEDIANTE APIS.	Cuadro de tipos de modelos con un caso de uso por tipo; diagrama de la arquitectura del resumidor; justificación con dos criterios	1.1 · 1.2 · 1.3
AE2. UTILIZAR UN MODELO DE IA A TRAVÉS DE API REST UTILIZANDO SOLICITUDES HTTP Y AUTENTICACIÓN, BASÁNDOSE EN LA DOCUMENTACIÓN OFICIAL DE OPENAI Y HUGGING FACE.	Enlace a <code>cliente_ia.py</code> y a sus pruebas; captura de <code>pytest</code> en verde; explicación de los campos de solicitud y respuesta con enlace a la documentación oficial	2.1 · 2.2 · 2.3
AE3. FORMULAR PROMPTS EFECTIVOS PARA MODELOS GENERATIVOS EN ESCENARIOS ZERO-SHOT O FEW-SHOT, CONSIDERANDO LA TAREA SOLICITADA Y LOS PARÁMETROS DE GENERACIÓN DISPONIBLES.	Prompts zero-shot y few-shot finales; tabla de iteraciones con cambio y efecto; comparación de temperature	3.1 · 3.2 · 3.3
AE4. IMPLEMENTAR UN PIPELINE DE PREPROCESAMIENTO Y EVALUACIÓN BÁSICA DE TEXTO EN PYTHON, APLICANDO TÉCNICAS DE LIMPIEZA, NORMALIZACIÓN Y MÉTRICAS INICIALES DE CALIDAD, DE ACUERDO CON LOS REQUISITOS DE ENTRADA Y SALIDA DEL MODELO.	Notebook con <code>limpiar</code> , <code>tokenizar</code> y <code>ngramas</code> aplicados a los 5 tickets; <code>resultados.csv</code> ; recomendación con números	4.1 · 4.2 · 4.3

Cuidado con las claves: el repositorio tiene un `.gitignore` con `.env`; ninguna clave de API aparece en código, notebooks, capturas ni historial de commits. Una clave publicada se revoca de inmediato y se registra en la bitácora.

Elemento 4 · Cierre como síntesis (plantilla)

1. ¿Qué configuración recomendé para el resumidor y con qué evidencia? 2. ¿Qué error me costó más (en el código, el prompt o la métrica) y cómo lo resolví? 3. Autodiagnóstico final comparado con el inicial: el cambio más grande.
4. Próximo paso: qué agregaría al resumidor en el módulo siguiente del plan (orquestración y agentes) y por qué.

Elemento 5 · Plataforma de publicación

Propuesta: repositorio en **GitHub** con el código y un sitio en **GitHub Pages** que funciona como portada del portafolio, generado desde una plantilla del curso. El plan ya incluye Git y GitHub entre los recursos del módulo, así que el portafolio usa la misma herramienta con la que se trabaja. El evaluador abre la URL sin iniciar sesión.

PENDIENTE: la plataforma la confirma cada institución. Lo que no cambia: un enlace que se abre sin credenciales.

Elemento 6 · Instrumento evaluativo asociado: rúbrica del portafolio

Se adjunta en la sección V del Anexo 2. Escala por criterio de 1 a 4. **Puntaje máximo:** 24.

Criterio	4 · Logrado	3 · Mayormente logrado	2 · Parcialmente logrado	1 · No logrado
Compleitud	Los 6 elementos presentes y con contenido propio	Falta desarrollar uno	Faltan dos o tres	Faltan más de tres
Evidencias AE1	Cuadro, diagrama y justificación con dos criterios	Dos de las tres	Una o sin justificación	Sin evidencia
Evidencias AE2	Código que se ejecuta, pruebas en verde y campos explicados con la documentación	Código correcto con explicación incompleta	Código que requiere la clave en el archivo o sin pruebas	Sin código
Evidencias AE3	Dos estrategias, tres o más iteraciones y comparación de temperature	Iteraciones incompletas	Una estrategia sin registro	Sin evidencia
Evidencias AE4	Pipeline aplicado, métricas bien calculadas y recomendación con números	Métricas correctas sin recomendación	Métricas mal calculadas	Sin evidencia

Criterio	4 · Logrado	3 · Mayormente logrado	2 · Parcialmente logrado	1 · No logrado
Reflexión, comunicación y seguridad	Introducción y cierre conectados, autodiagnóstico comparado; ninguna clave expuesta	Reflexión clara sin comparación	Reflexión genérica	Sin reflexión o con una clave expuesta

La nota se calcula con la escala de `B2-instrumentos.md` (exigencia 60 %).

PF1822 · Módulo 2 · B4 — Retroalimentación y aprendizaje colaborativo

Estado: borrador · **Va en:** Anexo N°2, sección V · **Guía:** Anexo N°7, num. 5.4

Para el 7,º: feedback + autoevaluación + coevaluación. La guía pide además la bitácora de registro de resultados y plan de trabajo.

- Instrumento evaluativo coherente seleccionado
- Instrumento de autoevaluación diseñado
- Instrumento de coevaluación diseñado
- Bitácora de registro de resultados y plan de trabajo diseñada

a) Mecanismo de retroalimentación

Qué	Cómo
Instrumento base	La rúbrica de revisión de la solución (B2-instrumentos.md , instrumento 1): nivel por criterio, no solo nota.
Plazo	Máximo 2 días hábiles desde cada entrega.
Formato	Comentarios en el código (revisión del <i>pull request</i> o comentarios en el notebook) + un párrafo de síntesis en el LMS.
Estructura	<i>Una fortaleza</i> (con la línea o la decisión concreta), <i>una mejora</i> (con el indicador) y <i>un siguiente paso</i> .
Reentrega	La actividad 1 admite una reentrega en la semana; se registra en la bitácora.
Feedback automático	pytest da retroalimentación inmediata al participante; el notebook guiado (R05) tiene celdas que se autocorrijen; la prueba objetiva y el video interactivo explican cada respuesta.

Ejemplo de comentario de nivel esperado (actividad 2):

Fortaleza: tu limpiar quita bien la firma y reemplaza el correo por [correo] , así que no envías datos personales al modelo. *Mejora:* tu ROUGE-L de T1 da 0,40 y el cálculo de comprobación da 0,733 (indicador 4.3): estás usando el tokenizador por defecto de rouge-score, que corta las palabras con tilde. *Siguiente paso:* usa TokenizadorES y vuelve a calcular la tabla antes de escribir tu recomendación.

b) Pauta de autoevaluación

La completa el participante antes de cada entrega. Escala: 1 = no lo logré, 2 = con ayuda, 3 = solo, pero con dudas, 4 = solo y lo puedo explicar.

#	Afirmación	AE	1	2	3	4
1	Puedo explicar qué componente de mi arquitectura guarda la clave y por qué	AE1				
2	Clasifiqué los tipos de modelos con un caso de uso para cada uno	AE1				
3	Mis solicitudes se autentican con variables de entorno, nunca con la clave en el código	AE2				
4	Mis pruebas pasan sin conexión a internet	AE2				
5	Mis prompts tienen rol, restricciones y formato de salida	AE3				
6	Registré cada cambio de prompt con su efecto	AE3				
7	Mi limpieza quita ruido y datos personales sin borrar el problema del ticket	AE4				
8	Comprobé mi métrica con el cálculo de referencia antes de usarla	AE4				

Pregunta abierta: ¿qué decisión de diseño cambiarías si el resumidor tuviera que procesar 10 veces más tickets?

c) Pauta de coevaluación

En parejas asignadas por el tutor: cada participante clona el repositorio de su compañero, ejecuta las pruebas y el notebook con los mismos tickets, y responde en el foro. Se valora la calidad del aporte, no la nota del compañero.

#	Criterio	Sí	En parte	No	Evidencia o comentario
1	Las instrucciones del README bastan para ejecutar el proyecto				
2	Las pruebas pasan sin conexión y sin clave real				
3	No hay ninguna clave en el código, el notebook ni el historial				
4	Con los mismos tickets obtengo métricas parecidas a las que reporta				
5	La recomendación final se sostiene con su tabla				

Cierre de la coevaluación: una práctica que voy a adoptar de tu proyecto y una sugerencia concreta para mejorarlo.

d) Bitácora de registro de resultados y plan de trabajo

Plantilla única para todo el módulo (archivo `BITACORA.md` en el repositorio o página del portafolio), actualizada al cerrar cada actividad y revisada por el tutor en cada devolución.

Parte 1 • Registro de resultados

Fecha	Actividad	Indicadores	Resultado	Problema encontrado (síntoma → causa → solución)	Evidencia
2026-10-15	Actividad 2, paso 4	4.3	Rúbrica: criterio 6 = nivel 2	ROUGE-L de $T1 = 0,40$ → el tokenizador por defecto cortaba "sesión" y "facturación" → <code>TokenizadorES</code> ; ahora da 0,733	Commit con la corrección

Parte 2 • Plan de trabajo

Indicador por mejorar	Acción concreta	Recurso de apoyo	Plazo	¿Logrado?
4.3	Recalcular la tabla completa y rehacer la recomendación	Cápsula 4, lámina 8	Antes del proyecto final	

La bitácora es evidencia del criterio 6 de la rúbrica en la actividad 2.

PF1822 · Módulo 2 · C1, C3 y C5 — Metodología, motivación y habilidades del siglo XXI

Estado: borrador · **Va en:** Anexo N°2, sección VI a), d) y e) · **Guía:** Anexo N°7, num. 7

Para el 7.º: C1 binario (la metodología está enfocada en la competencia de ESTE módulo); C3 binario (el módulo aporta al aprendizaje a través de la interacción con la plataforma); C5, tres estrategias para tres habilidades del siglo XXI.

Este texto es la base común. Cada institución lo adapta a su LMS, su tutor y su calendario en `propuestas/<cliente>/PF1822/anexo2.md`, sin cambiar el anclaje a la competencia.

a) ¿Qué hará? · Estrategia metodológica (C1)

Competencia del módulo (textual): IMPLEMENTAR LA ARQUITECTURA BÁSICA DE UNA APLICACIÓN QUE INTEGRE MODELOS DE IA MEDIANTE APIS Y PREPROCESAMIENTO DE TEXTO PARA CONSUMO POR MODELOS DE LENGUAJE, GARANTIZANDO EL CONSUMO EFICIENTE Y LA EVALUACIÓN DE RESULTADOS EN UN ENTORNO DE DESARROLLO DE SOFTWARE

La estrategia es **aprendizaje basado en proyectos sobre una aplicación que se construye por capas**. La competencia pide *implementar la arquitectura básica de una aplicación, con preprocesamiento, consumo eficiente y evaluación de resultados*; por eso el módulo no enseña APIs, prompts y NLP como temas separados, sino como las cuatro capas de un mismo producto: el resumidor de tickets de Nube Sur, una empresa ficticia. Primero se diseña su arquitectura (AE1), después se programa y prueba el cliente que consume el modelo (AE2), luego se construyen y afinan los prompts (AE3) y al final se agrega el pipeline que limpia la entrada y mide la calidad de la salida (AE4). El *consumo eficiente* se trabaja con datos: tokens por solicitud, limpieza antes de enviar y derivación sin llamar al modelo cuando no hace falta. Cada tramo combina una cápsula, práctica guiada en el notebook y práctica autónoma en el repositorio del participante, revisada por el tutor antes del tramo siguiente.

Tramo	Actividades del módulo	Horas	Sincrónica	Asincrónica
1 · AE1	Video de bienvenida, autodiagnóstico, cápsula 1, cuadro de familias de modelos, diagrama del resumidor; sesión en vivo: diseño de arquitectura en grupo	4	1,5	2,5
2 · AE2	Cápsula 2, notebook guiado (secciones 0 a 6), actividad 1 parte B, pruebas unitarias; sesión en vivo: revisión de código	6	1,5	4,5
3 · AE3	Cápsula 3, video interactivo, actividad 2 pasos 2 y 3; sesión en vivo: taller de prompts	5	1,5	3,5
4 · AE4	Cápsula 4, notebook guiado (secciones 7 a 9), actividad 2 pasos 1, 4 y 5, tablero, coevaluación, prueba objetiva, proyecto final, portafolio; sesión en vivo: lectura de métricas	6	1,5	4,5
Total		21	6	15

Rol del tutor académico. En lo teórico, presenta cada tramo y conduce la sesión en vivo sobre el caso. En lo práctico, revisa el código de cada participante con la rúbrica y comentarios en línea, devuelve en máximo 2 días hábiles, mantiene el tablero del laboratorio, responde el foro y da asistencia técnica con el entorno de Python y las claves de práctica.

Atención a la diversidad. Tres entornos posibles (local, Colab o contenedor) para que el equipo del participante no sea una barrera; cada contenido en más de un formato (cápsula con transcripción, notebook con celdas autocorregibles, video con preguntas); el autodiagnóstico inicial permite al tutor reforzar Python antes del tramo 2 a quien lo necesite.

b) ¿Cómo lo hará? · Actividades didácticas

Dos actividades prácticas distintas, desarrolladas en `C2-actividades.md`:

1. **Un cliente de API para el resumidor** — resolución de problemas, con el notebook guiado y las cápsulas como apoyo. Respuesta modelada: diagrama, `cliente_ia.py` y sus pruebas.
2. **Laboratorio de prompts y métricas** — análisis de caso en forma de simulación de una decisión real, con tablero de puntajes e insignias. Respuesta modelada: pipeline, prompts, registro de iteraciones, cálculo de comprobación y recomendación.

c) ¿Con qué lo van a hacer? · Medios

Enlace al LMS: **PENDIENTE**: por institución (va aquí, dentro de esta respuesta). Medios del módulo: video de bienvenida (R01), infografía de la ruta (R02), cuatro cápsulas narradas (R03), cuadro comparativo de familias de modelos (R04), **notebook guiado** (R05) y **video interactivo** (R06) —las dos herramientas didácticas de C4—, plantillas de bitácora y portafolio, Python, Git y GitHub como entorno de práctica.

d) Aspectos motivacionales (C3)

- **Un producto que crece.** El mismo resumidor gana una capa por tramo; el participante ve su aplicación funcionar desde la segunda semana.
- **Ruta visible y navegación consistente.** Infografía y barra de progreso con las cuatro estaciones; cada sección del LMS usa el ícono de su estación.
- **Retroalimentación inmediata.** Pruebas que se ponen en verde, celdas que se autocomprueban, preguntas del video con respuesta al instante.
- **Tablero e insignias.** El laboratorio publica el mejor ROUGE-L de cada participante; hay insignias por resultado y por proceso.
- **Un repositorio para mostrar.** El portafolio y el código quedan públicos para un empleador.

e) Habilidades del siglo XXI (C5)

Tres habilidades de la lista del Anexo N°7 (pág. 111), cada una con su estrategia y evidencia.

#	Habilidad transversal	Estrategia para desarrollarla	Cómo se evidencia
1	Pensamiento crítico (maneras de pensar)	En el laboratorio, la configuración del resumidor se decide con la tabla de métricas y no con la impresión; el participante debe explicar un caso en que la métrica y su propia lectura no coinciden.	Recomendación de la actividad 2 con números y el análisis del caso de desacuerdo, evaluados en el criterio 6 de la rúbrica.
2	Colaboración (maneras para trabajar)	Revisión de código entre pares: cada participante clona el repositorio del otro, ejecuta pruebas y notebook, y deja sugerencias que el autor responde.	Pauta de coevaluación en el foro y respuesta del autor con los cambios aplicados.
3	Responsabilidad personal y social (habilidades para vivir)	Cada decisión del módulo se revisa también desde la seguridad y la ética: no exponer claves, no enviar datos personales al modelo, no presentar como hecho lo que el modelo inventa.	La limpieza reemplaza correos y enlaces; ninguna clave en el repositorio; la restricción anti-inventación en los prompts; criterio "Reflexión, comunicación y seguridad" de la rúbrica del portafolio.

C1 · control antigenérico: si este texto sirviera para otro plan formativo cambiando solo el nombre de la herramienta, está mal. Aquí se nombran la competencia textual, la arquitectura, los endpoints, los prompts, las métricas y el caso de ESTE módulo.